Copa+: Analysis and Improvement of the Delay-based Congestion Control Algorithm Copa

Wanchun Jiang*, Haoyang Li*, Zheyuan Liu*, Jia Wu*, Jiawei Huang*, Danfeng Shan† and Jianxin Wang*
*School of Computer Science and Engineering, Central South University, Changsha, China
†School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China
jiangwc@csu.edu.cn

IEEE INFOCOM 2022 - IEEE Conference on Computer Communications | 978-1-6654-5822-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/INFOCOM48880.2022.9796913

***Abstract*—Copa is a delay-based congestion control algorithm proposed in NSDI recently. It can achieve consistent high performance under various network environments and has already been deployed in Facebook. In this paper, we theoretically analyze Copa and reveal its large queuing delay and poor fairness issue under certain conditions. The root cause is that Copa fails to clear the bottleneck buffer occupancy periodically as expected. Accordingly, Copa may get a wrong base RTT estimation and enter its competitive mode by mistake, leading to large delay and unfairness. To address these issues, we propose Copa+, which enhances Copa with a parameter adaptation mechanism and an optimized competitive mode entrance criterion. Designed based on our theoretical analysis, Copa+ can adaptively clear the bottleneck buffer occupancy for correct estimation of base RTT. Consequently, Copa+ inherits the advantages of Copa but achieves lower queuing delay and better fairness under different environments, as confirmed by the real-world experiments and simulations. Specifically, Copa+ has the highest throughput similar to Copa but 11.9% lower queuing delay over different Internet links among different cloud nodes, and achieves 39.4% lower queuing delay and 8.9% higher throughput compared to Sprout over emulated cellular links.**

***Index Terms*—Delay-based Congestion Control, Copa, Cubic**

I. Introduction

A good end-to-end congestion control (CC) algorithm for the Internet should achieve consistent high performance, including high throughput, low queuing delay, fairness, and TCP friendly. Typically, CC algorithms, from Reno [1] to and Cubic [3], rely on packet loss as the fundamental congestion signal and fill the buffer for high throughput. However, with the rapid increase of the link capacity, the impact of delay on user-experience becomes more and more significant [5], [6]. The large queuing delay incurred by the loss-based CC algorithms become intolerable to many delay-sensitive applications, such as online meeting and interactive web applications, and the buffer-bloat problem [8] becomes a hot research topic. The delay-based CC is the elixir for low queuing delay, as the delay signal is easy to obtained and fine-gained enough to reflect the degree of congestion.

However, the traditional delay-based CC algorithms such as Vegas [4] and TCP Fast [7] suffer from wrong delay estimation [9] and poor throughput when coexisting with the loss-based algorithms. To overcome the issues of the delay-based CC, the Copa algorithm proposed in NSDI recently periodically clears the bottleneck buffer occupancy for delay estimation and involves a competitive mode for TCP friendliness [9]. In this way, compared to Cubic [3] and BBR [16], Copa is shown to have similar throughput, but much lower delay, and fairer with diverse RTTs. It also co-exists well with Cubic, and is robust to random packets loss and large bottleneck buffers [9]. Moreover, Copa can also achieve good performance similar to the representative CC algorithms under specialized network environments or workloads, such as datacenters [10] and cellular networks [27]. Furthermore, compared to the learning-based algorithms, such as PCC [11] designed for consistent high performance under different network environments, Copa can achieve lower delay and be fairer [9]. Due to these excellent features, Copa has already been deployed in production [25], [26].

In this paper, we conduct theoretic analysis on Copa and reveal that Copa fails to periodically clear the bottleneck buffer occupancy as expected under certain conditions. Accordingly, Copa will make wrong delay estimation and enter its competitive mode by mistake, leading to large queuing delay and unfairness. These problems are significant in realistic network conditions such as a large number of flows, small bandwidth, or small propagation delay, as confirmed by simulations.

To address these problems, we design Copa+ as a step further. In detail, guided by the theoretical result, we reveal that the underlying reason for the issues of Copa is the hard-wire of its target queuing delay and rate adjustment granularity. Therefore, Copa+ decouples them and searches for a proper rate adjustment granularity parameter according to the cubic function such that the bottleneck buffer occupancy could be emptied periodically. Excepting the parameter adaptation mechanism, Copa is also enhanced with an optimized competitive mode entrance criterion. In combination, Copa+ can adaptively clear the bottleneck buffer occupancy under different network environments. Accordingly, Copa+ can achieve consistent low latency and fairness, while inheriting the advantages of Copa.

We evaluate Copa+ with experiments based on an implementation in the user-space of Linux and simulations on NS3 [24]. Real experiments over various links among different cloud nodes show that Copa+ has the highest throughput similar to Copa but 11.9% lower queuing delay. The experiment results with the emulated cellular links also confirm that Copa+ achieves 39.4% lower queuing delay and 8.9% higher throughput compared to Sprout [27].

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

II. Background and Related Work

A. Development of Congestion Control

Nowadays, many CC algorithms have been developed with the evolution of the network environment. They can be briefly classified into the following three categories.

Loss-based. Traditionally, CC algorithms mainly fill up the bottleneck buffers for high throughput and the packet loss is usually adopted as the congestion signal. Wherein, Compound TCP [2] and TCP Cubic [3] are employed in reality and TCP Cubic is the dominant algorithm over the Internet nowadays. Wherein the Cubic function is famous for helping CC algorithm to rapidly probe for suitable congestion window under different environments.

The major problem of the loss-based CC algorithm is the large queuing delay, coming along with the fulfilled bottleneck buffer occupancy. To solve this intrinsic "buffer-bloat" problem, the Active Queue Management (AQM) [30]–[32] mechanisms suggest randomly dropping or marking some packets before the bottleneck buffer overflows. But the AQM mechanism is always hard to deploy in practice, because of the need to modify intermediate nodes in network and the difficulty of tuning parameters in diverse environments.

Delay-based CC. The root cause of the "buffer-bloat" problem is the use of packet loss as the congestion signal. Therefore, delay-based CC algorithms, such as Vegas [4] and TCP Fast [7], suggest replacing the packet loss signal with the delay signal. Compared with the loss signal, delay signal has the following advantages. (1) The delay signal could be gained easily without modifying any intermediate nodes in network. (2) In random loss (like wireless network) or no loss (like data center network) environment, it is difficult to use loss signal for the judgement on whether the network is in congestion or not. While the delay signal can still correctly reflect the congestion status. (3) In contrast to one-bit signals such as loss signal and ECN, the delay signal provides fine-grained information about the extent of congestion. This is helpful for CC algorithms to precisely adjust the sending rate. (4) Delay is not only the congestion signal, but also the performance index of CC algorithms, which should be directly controlled. However, traditional delay-based CC algorithms often suffer from the following issues. First of all, they rely on the correct delay estimation, which is hard in reality due to the factors such as ACK compression [9], [28], network jitter and the late-comer problem [29]. Second, traditional delay-based CC algorithms always suffer from lower throughput, when they coexist with the mainstream loss-based CC algorithms over the Internet.

In recent years, many delay-based CC algorithms are proposed because of the emergence demands on low latency. For instance, Google designs GCC [20] to simultaneously provide high throughput for a good quality of the transferred images and low latency for the instantaneity in the environment of Web Instant Messaging. Moreover, the delay-based CC algorithms TIMELY [15], DX [21], Swift [22] are proposed for the data center networks, where ultra-low latency and high bandwidth are demanded. Furthermore, the BBR [16] and Copa [9] algorithms proposed recently exhibits that the delay-based CC can also accommodate diverse network environments spontaneously. Specifically, Copa addresses the above issues by periodically clearing the bottleneck buffer occupancy for correct delay estimation and involving a competitive mode for TCP friendliness and is shown to outperform a large number of existing CC algorithms [9].

Learning-based CC. On the other hand, there are viewpoints that the loss-based and delay-based CC algorithms are indeed designed based on certain implicit assumptions of the network model. Moreover, the network is too complicated to model and the design of CC is hard for human engineering. Hence, many learning-based CC algorithms are proposed recently, including Remy [13], [14], PCC [11], [17], [19], Indigo [23] and Orca [18]. Their rate regulations are based on the model trained from historical data without any assumption of the network model, and thus could achieve good performance under different network environments. But the learning-based CC algorithms are always hard to understand, have a high implementation overhead [18] and suffer from convergence problems [35].

B. Brief of Copa

Copa was designed by Venkat Arun and Hari Balakrishnan. It works as follows. On every ACK, the sender sets its target rate to δ^{-1}/d_q, calculates the current rate by W/RTT_{stand}, and then adjusts the congestion window W by:

$$W \leftarrow \begin{cases} W + \dfrac{V \cdot \delta^{-1}}{W}, if \ \dfrac{W}{RTT_{stand}} \leq \dfrac{\delta^{-1}}{d_q} \\ W - \dfrac{V \cdot \delta^{-1}}{W}, if \ \dfrac{W}{RTT_{stand}} > \dfrac{\delta^{-1}}{d_q} \end{cases} \tag{1}$$

Where δ^{-1} is a parameter with unit $pkts$ and d_q is the queuing delay estimated by $RTT_{stand} - RTT_{min}$. Here RTT_{stand} is the smallest RTT observed over a time of $sRTT/2$, which eliminates the impacts of ACK compression and network jitter [9]. Here $sRTT$ is the smoothed RTT and RTT_{min} is the smallest RTT observed over a long time (10 seconds by default). Obviously, RTT_{min} is the estimation of the physic propagation delay RTT_{base}. Moreover, V is a velocity parameter used to speed up the convergence phase. The initial value of V is 1. If W is adjusted in the same direction for 3 RTTs, V will be doubled. Or else, V is reset to 1.

Theoretically, Copa maintains the periodical behavior shown in Fig. 1 for the measurement of RTT_{min}. In detail, Copa converges to the equilibrium point where the queue size is δ^{-1} [9]. Around this equilibrium point, if the queuing delay increases to be larger than δ^{-1}/C at point A, the target rate will become smaller than the current rate. Since RTT_{stand} is the minimum RTT observed in $sRTT/2$, the sender takes $sRTT/2$ to estimate RTT_{stand}. In other words, the sender knows the variation of queue size $sRTT/2$ later and then starts to decrease W. Subsequently, the sender would observe the peak of queuing delay (point B) one RTT later. In contrast, when the queuing delay is below δ^{-1}/C, the

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

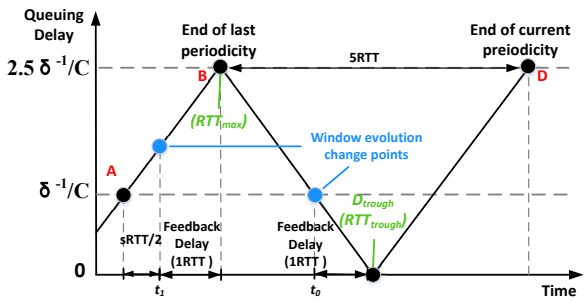


Fig. 1. Evolution of the queuing delay observed at the sender in Copa

sender will increase W. Because RTT is always the smallest value in $sRTT/2$ during the descending process, the sender doesn't need $sRTT/2$ to estimate RTT_{stand}. Consequently, the queuing delay will reach the minimum value D_{trough} after one RTT feedback delay. According to equation (1), W decreases by δ^{-1} packets per RTT, i.e., the queuing delay will decrease from δ^{-1}/C to zero. In other words, D_{trough} will be close to 0 with only one flow. Note that the queuing delay can only keep growing or reducing for no more than 2.5 RTTs, which is smaller than the interval of doubling V, i.e., 3 RTTs. Therefore, $V = 1$ around the equilibrium point. Repeating these behaviors, Copa can clear the buffer occupancy every 5 RTTs such that the RTT_{min} is measured correctly and the late-comer problem is avoided. In total, the periodical behavior of Copa is the basis of correct delay estimation.

On the other hand, Copa enters into the competitive mode for high throughput when co-existing with buffer filling flows. The criterion of coexisting with buffer filling flows is that the buffer occupancy is not emptied in the last 5 RTTs, i.e., the following inequality doesn't hold.

$$d_q < 0.1(RTT_{max} - RTT_{min}) \tag{2}$$

where RTT_{max} is the maximum RTT observed in the recent $4RTTs$. In the competitive mode, Copa will perform AIMD on δ^{-1} based on packet success or loss for high throughput, instead of setting $\delta^{-1} = 2$ as in the default mode. In the competitive mode, Copa will switch to the default mode once observing an empty buffer, i.e., when inequality (2) holds. With this competitive mode, Copa is TCP friendly [9].

III. Analysis of Copa

In this section, we model and analyze Copa, which periodically clears the buffer occupancy for correct delay estimation and good performance. The results show that the buffer occupancy may not be cleared periodically as expected under certain conditions. Accordingly, Copa may wrongly estimate RTT_{base} and enter its competitive mode by mistake, leading to large queuing delay and unfairness, as confirmed by simulation results based on NS3. To avoid confusion, the symbols of delays are listed in Table. I.

TABLE I
Symbols of Delays

Symbol	Definition
RTT_{base}	The physic link propagation delay.
RTT_{min}	The smallest RTT in a long period (10s).
RTT_{trough}	The minimum RTT in the cycles of Copa.
RTT_{max}	The maximum RTT in the cycles of Copa.
$RTT(t)$	The round trip time at time t.
$srtt$	EWMA of $RTT(t)$ in TCP.
$RTT_{stand}(t)$	The minimum RTT observed in $sRTT/2$.
$d_q(t)$	Queuing delay, equals $RTT_{stand}(t) - RTT_{min}$.
D_{trough}	The minimum d_q, equals $RTT_{trough} - RTT_{min}$

A. Modeling

Whether D_{trough} can periodically fulfill inequality (2) is important for the estimation of RTT_{base} and the performance of Copa. The literature [9] only analyzes Copa with one flow, as shown in Fig. 1. Here, we establish a mathematical model with multiple Copa flows. Assuming N synchronous Copa flows sharing a single bottleneck link with capacity C, we have the following theorem.

Theorem 1 *Assume RTT_{min} is the measured value of the link propagation delay RTT_{base}, the minimum queuing delay D_{trough} in the cycles of Copa can be expressed by*

$$D_{trough} = \sqrt{(\frac{RTT_{min}}{2})^2 + (\frac{N\delta^{-1}}{C})^2} - \frac{RTT_{min}}{2} \tag{3}$$

Proof *When a packet is sent out but without receiving the ACK, this outstanding packet would be under one of the three conditions: (i) the packet is on the forward link; (ii) the packet is in the bottleneck link buffer; (iii) the ACK of this packet is on the backward link. On the one hand, the total number of outstanding packets equals to $C \cdot RTT(t)$. On the other hand, the number of outstanding packets is limited by the congestion window one RTT ago $N \cdot W(t - RTT(t))$. In other words, the following equation always holds.*

$$RTT(t) = N \cdot W(t - RTT(t))/C \tag{4}$$

Assume the current sending rate equals the target rate at t_0:

$$\frac{W(t_0)}{RTT_{stand}(t_0)} = \frac{\delta^{-1}}{d_q(t_0)} \tag{5}$$

where $d_q(t_0) = RTT_{stand}(t_0) - RTT_{min}$. If the current rate is larger than the target rate before the time t_0, W will keep decreasing before t_0 according to the Copa algorithm, as shown in Fig. 1. According to (4), RTT decreases with the reduction of the W. Hence, $RTT(t_0)$ is the smallest one in the recent $sRTT/2$ time interval, i.e.,

$$RTT_{stand}(t_0) = RTT(t_0) \tag{6}$$

In Copa, W is decreased by $V\delta^{-1}$ in one RTT, i.e.,

$$W(t_0) = W(t_0 - RTT(t_0)) - V\delta^{-1} \tag{7}$$

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

As Copa claimed in [9], $V = 1$ around the equilibrium point. Combining (4), (5), (6) *and* (7) *and eliminating d_q and $RTT_{stand}(t_0)$, we get*

$$\begin{cases} RTT(t_0) = N \cdot W(t_0 - RTT(t_0))/C \\ \dfrac{W(t_0)}{RTT(t_0)} = \dfrac{\delta^{-1}}{RTT(t_0) - RTT_{min}} \\ W(t_0) = W(t_0 - RTT(t_0)) - \delta^{-1} \end{cases} \tag{8}$$

Eliminating $RTT(t_0)$ and $W(t_0 - RTT(t_0))$ in (8)*, we have*

$$W(t_0) = \sqrt{(\frac{RTT_{min} \cdot C}{2N})^2 + \delta^{-2}} + \frac{RTT_{min} \cdot C}{2N} \tag{9}$$

Because W keeps decreasing before t_0 and keeps increasing after t_0, $W(t_0)$ is a minimum point. RTT would also reach the minimum value RTT_{trough} after one RTT from t_0, as shown in Fig. 1. According to (4)*,*

$$RTT_{trough} = N \cdot W(t_0)/C \tag{10}$$

Substituting (9) *into* (10) *and referring $D_{trough} = RTT_{trough} - RTT_{min}$, we can get (3).* ■

From **Theorem 1**, we could infer that D_{trough} is influenced by RTT_{min}, N, and C, where RTT_{min} is the estimation of RTT_{base}. To further demonstrate the impacts of these three factors, we deduce the partial derivative of D_{trough} with respect to RTT_{base}, C and N, respectively.

- $RTT_{min} = RTT_{base} + \epsilon$, where ϵ is the difference between the estimated value RTT_{min} and the real value RTT_{base}. The partial derivative of D_{trough} with respect to RTT_{base} is

$$\frac{\partial D_{trough}}{\partial RTT_{base}} = \frac{\partial D_{trough}}{\partial RTT_{min}} \cdot \frac{\partial RTT_{min}}{\partial RTT_{base}} = \frac{\partial D_{trough}}{\partial RTT_{min}} \tag{11}$$

  Substituting (3) into (11), we get

$$\frac{\partial D_{trough}}{\partial RTT_{base}} = \frac{\frac{RTT_{min}}{2} - \sqrt{(\frac{RTT_{min}}{2})^2 + (\frac{N \cdot \delta^{-1}}{C})^2}}{2\sqrt{(\frac{RTT_{min}}{2})^2 + (\frac{N \cdot \delta^{-1}}{C})^2}} < 0 \tag{12}$$

  Thus, the relation between D_{trough} and RTT_{base} is monotonically decreasing.
- According to (3), the partial derivative of D_{trough} with respect to C is

$$\frac{\partial D_{trough}}{\partial C} = \frac{-\frac{N^2 \delta^{-2}}{C^3}}{\sqrt{(\frac{RTT_{min}}{2})^2 + (\frac{N \cdot \delta^{-1}}{C})^2}} < 0 \tag{13}$$

  Hence, the relation between D_{trough} and C is monotonically decreasing.
- Similarly, the partial derivative of D_{trough} with respect to N is

$$\frac{\partial D_{trough}}{\partial N} = \frac{N(\frac{\delta^{-1}}{C})^2}{\sqrt{(\frac{RTT_{min}}{2})^2 + (\frac{N \cdot \delta^{-1}}{C})^2}} > 0 \tag{14}$$

  Hence, the relation between D_{trough} and N is monotonically increasing.

In total, D_{trough} is proportional to N, and inversely proportional to C and RTT_{base}. It indicates that Copa may fail to observe the empty buffer occupancy for correct measurement of RTT_{base} as expected and suffer from performance degradation under certain conditions, as discussed later.

B. Problem of Copa

In Copa, D_{trough} is the local minimum queuing delay which reflects whether the buffer occupancy can be cleared or not in each cycle. When D_{trough} is far away from 0, i.e., inequality (2) doesn't hold, the queue is cumulative and accordingly Copa would overestimate RTT_{base}, resulting in large queuing delay and unfairness. In fact, based on **Theorem 1**, we have the following corollary.

Corollary 1 *Copa will fail to periodically clear the buffer occupancy, i.e., inequality (2) doesn't hold in each cycle, for the correct measurement RTT_{base}, when*

$$3.75N\delta^{-1} \geq C \cdot RTT_{min} \tag{15}$$

Proof *Similar to the deduction of RTT_{trough}, we could deduce the expression of RTT_{max} as follows. Assume that the current rate is smaller than the target rate before the time t_1, and the current rate equals the target rate at time t_1, as shown in Fig. 1. Before t_1, Copa will keep increasing W. According to* (4)*, RTT increases with the growth of the W. Therefore, the $RTT_{stand}(t_1)$ equals $RTT(t_1 - srtt/2)$ in the recent $srtt/2$.*

$$\begin{cases} \dfrac{W(t_1)}{RTT_{stand}(t_1)} = \dfrac{\delta^{-1}}{d_q(t_1)} \\ RTT_{stand}(t_1) = RTT(t_1 - srtt/2) \end{cases} \tag{16}$$

Substituting $t = t_1 - srtt/2$ into (4)*, we get*

$$RTT(t_1 - \frac{srtt}{2}) = W(t_1 - \frac{srtt}{2} - RTT(t_1 - \frac{srtt}{2})) \cdot \frac{N}{C} \tag{17}$$

In Copa, the congestion window W grows $V\delta^{-1}$ in one RTT. Thus, the variation of W in $srtt/2 + RTT(t_1 - srtt/2))$ is about $1.5V\delta^{-1}$, i.e.,

$$W(t_1) = W(t_1 - srtt/2 - RTT(t_1 - srtt/2)) + 1.5V\delta^{-1} \tag{18}$$

Similarly, combining (16)*,* (17) *and* (18)*, we have*

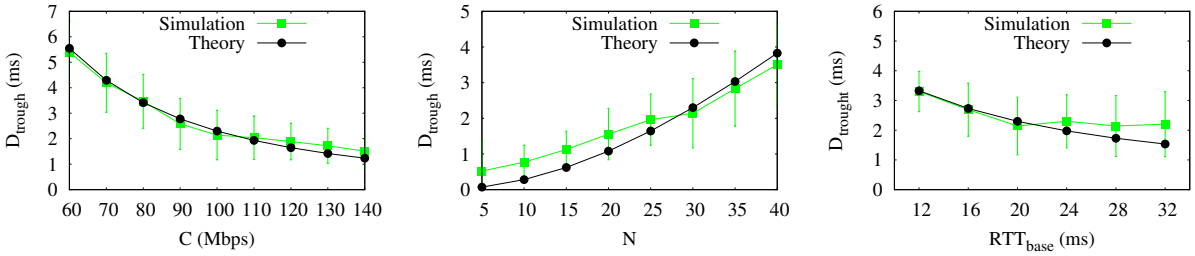
$$\begin{aligned} W(t_1) = & \sqrt{(\frac{RTT_{min} \cdot C + 2.5N\delta^{-1}}{2N})^2 - 1.5\delta^{-2}} \\ & + \frac{RTT_{min} \cdot C}{2N} + \frac{2.5\delta^{-1}}{2} \end{aligned} \tag{19}$$

Because W keeps increasing before t_1 and keeps decreasing after t_1, $W(t_1)$ is a maximum value. RTT would also reach the maximum value RTT_{max} after one RTT from t_1. According to (4)*,*

$$\begin{aligned} RTT_{max} = & \sqrt{(\frac{RTT_{min}}{2} + \frac{2.5N\delta^{-1}}{2C})^2 - 1.5(\frac{N\delta^{-1}}{C})^2} \\ & + \frac{RTT_{min}}{2} + \frac{2.5N\delta^{-1}}{2C} \end{aligned} \tag{20}$$

Combining (2)*,* (3) *and* (20)*, we can get the condition where D_{trough} is far away from* 0*, i.e.,* ***Corollary 1****.* ■

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.



(a) Impact of C (b) Impact of N (c) Impact of RTT_{base}

Fig. 2. The variation of D_{trough} with different C, N and RTT_{base} in Copa

In the Internet, when the number of concurrent flows is large, the inequality (15) in **Corollary 1** holds easily. When inequality (15) holds, Copa would enter the competitive mode even by mistake even without loss-based flows. Moreover, when inequality (15) holds, RTT_{base} may be overestimated, i.e., RTT_{min} becomes larger. Correspondingly, the measured delay d_q decreases, target rate δ^{-1}/d_q increases, and Copa will inject more packets into the bottleneck link, leading to large queuing delay. Furthermore, in the competitive mode, Copa will keep increasing δ^{-1} until there is packet loss. Correspondingly, the queuing delay becomes high.

In addition, when late-comers enter, Copa will become unfair. This is because late-comers will get a higher RTT_{min}, i.e., overestimate RTT_{base}, compared to original flows. Consequently, the target rate of late-comers would be larger than the original flows and the late-comers will take more bandwidth.

C. Verification

To verify the above analytical results, we implement Copa in NS3 and conduct the following simulations. First, the correctness of our Copa implementation is verified by the user-space implementation of Copa [9].

Next, we conduct simulations with a dumbbell topology involving 2 senders and 1 receiver to verify **Theorem 1**. By default, we set the number of flows $N = 30$, $C = 100Mbps$ and $RTT_{base} = 20ms$. All flows are evenly distributed across two senders. Specifically, we disable the competitive mode of Copa to avoid its impact on D_{trough}. The minimum RTT of each cycle, i.e., when d_q changes direction twice corresponding to period from point B to D in Fig. 1, is recorded as D_{trough}. We account for only the average D_{trough} of all flows during the first $10s$, where the initial well-measured RTT_{min} is kept unchanged in Copa. This is because Copa may get a wrong estimation of RTT_{base} after $10s$ and makes D_{trough} deviates from the theoretical result. As shown in Fig. 2, the simulation result manifests that D_{trough} coincides with the theoretical value calculated according to (3), with the variation of C, N and RTT_{base} based on the default configuration.

Subsequently, we turn to the results after $10s$. In detail, we record the RTT_{min} measured on the receiving of each ACK and calculate the average RTT_{min} in every second for all flows to verify **Corollary 1**. As shown in Fig. 3(a), when $C = 60Mbps$ with default N and RTT_{base}, the inequality (15) holds, and the average RTT_{min} after updating ($10s$) equals $24.90ms$. It indicates that Copa can not periodically clear the bottleneck buffer and get a wrong estimation of RTT_{base}. In contrast, when $C = 140Mbps$, the inequality (15) doesn't hold and Copa gets RTT_{min} correctly. Similarly, in Fig. 3(b) and (c), (15) still holds when $N = 40$ with default C and RTT_{base}, or $RTT_{base} = 12ms$ with default C and N. Under these conditions, we have $RTT_{min} = 22.36ms$ and $RTT_{min} = 12.72ms$, respectively. Inversely, Copa correctly estimates RTT_{base} when $N = 40$ or $RTT_{base} = 32ms$, which don't fulfill inequality (15). In other words, Copa fails to clear the queue when (15) holds. Therefore, **Corollary 1** is verified. Note that flows are asynchronous in simulations although the analytical results are deduced under the synchronization assumption.

In total, Copa wrongly estimates RTT_{base} with a large number of flows, small link bandwidth or small propagation delay, and may suffer from large queuing delay and unfairness.

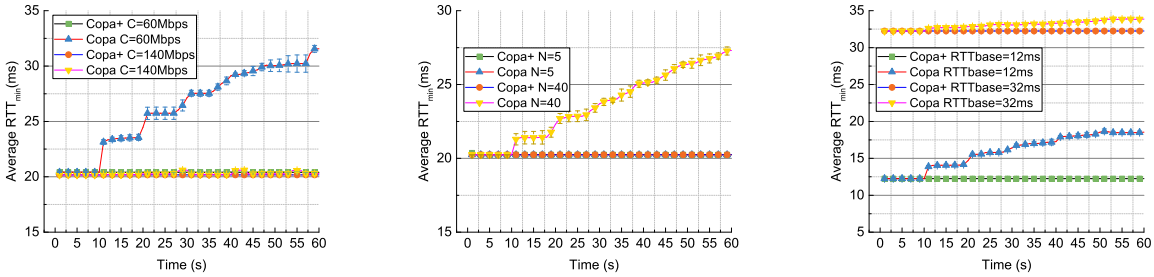
IV. Design of Copa+

In this section, we design Copa+, which enhances Copa to address the issues of large queuing delay and unfairness. First, based on the above analysis, we reveal that the underlying reason for these issues of Copa is the hard-wire of its target queuing delay and rate adjustment granularity. Furthermore, we decouple them such that the rate adjustment granularity can be adjusted adaptively. Meanwhile, the entrance criterion of competitive mode is enhanced.

A. Basic Idea

According to **Theorem 1**, δ^{-1} is the only tunable factor influencing the value of D_{trough}. However, the parameter δ^{-1} of Copa determines both the target queuing delay and the adjustment granularity of W. Because the target queuing delay is configured directly according to the user-experience, the change of δ^{-1} would greatly influence the performance of Copa. To avoid the modification of δ^{-1}, Copa+ introduces parameter λ to represent the adjustment granularity of the congestion window W and regulate λ adaptively such that D_{trough} fulfills inequality (2), as shown in the following corollary. In this way, RTT_{base} can be accurately estimated under different network environments.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.



(a) Impact of C (b) Impact of N (c) Impact of RTT_{base}

Fig. 3. Evolution of RTT_{min}

Corollary 2 *Under the same assumptions as* ***Theorem 1****, D_{trough} will decrease to zero with the increase of λ.*

Proof *With the adjustment granularity λ of the congestion window W, the equation* (7) *could be rewritten as*

$$W(t_0) = W(t_0 - RTT(t_0)) - \lambda \tag{21}$$

Accordingly, similar to (8)*, we have*

$$\begin{cases} RTT(t_0) = N \cdot W(t_0 - RTT(t_0))/C \\ \dfrac{W(t_0)}{RTT(t_0)} = \dfrac{\delta^{-1}}{RTT(t_0) - RTT_{min}} \\ W(t_0) = W(t_0 - RTT(t_0)) - \lambda \end{cases} \tag{22}$$

The solution to (22) *becomes*

$$\begin{aligned} W(t_0) = & \sqrt{(\frac{RTT_{min} \cdot C}{2N} + \frac{-\lambda + \delta^{-1}}{2})^2 + \lambda\delta^{-1}} \\ & + \frac{RTT_{min} \cdot C}{2N} + \frac{-\lambda}{2} \end{aligned} \tag{23}$$

As analyzed in section III, $W(t_0)$ is the minimum of W. For simplicity, let S denote the term $(RTT_{min} \cdot C)/N$ in (23) *and then the partial derivative of $W(t_0)$ with respect to λ is*

$$\frac{\partial W(t_0)}{\partial \lambda} = \frac{\frac{-S+\lambda+\delta^{-1}}{2} - \sqrt{(\frac{-S+\lambda+\delta^{-1}}{2})^2 + \lambda\delta^{-1}}}{2\sqrt{(\frac{-S+\lambda+\delta^{-1}}{2})^2 + \lambda\delta^{-1}}} \tag{24}$$

Since $\lambda\delta^{-1} > 0$, $\frac{\partial W(t_0)}{\partial \lambda}$ is negative. According to (10)*, $D_{trough} = N \cdot W(t_0)/C - RTT_{min}$. Thus, the partial derivative of D_{trough} with respect to λ is*

$$\frac{\partial D_{trough}}{\partial \lambda} = \frac{\partial W(t_0)}{\partial \lambda} \cdot \frac{N}{C} < 0 \tag{25}$$

Therefore, D_{trough} keeps decreasing with the increase of λ. In addition, according to (21)*, $W(t_0)$ can be zero with large λ and $D_{trough} = -RTT_{min}$ mathematically. In practice, it corresponds to the link under-utilization with a huge rate decrease. In other words, D_{trough} will decrease to zero with the increase of λ.* ■

Although large λ can make D_{trough} fulfill inequality (2), it might be too big such that D_{trough} is always zero, i.e., the throughput loss appears. In contrast, when λ is small, D_{trough} would always larger than zero. Consequently, Copa+ requires a proper value of λ. However, it's hard to theoretically calculate a precise value for λ, because the above analysis is based on the synchronization assumption while flows in real-world are generally asynchronous. Therefore, λ can only be probed heuristically. By the way, when each flow independently probes for a suitable λ, the λ of each flow should converge to the same final value for fairness.

B. Details of Copa+

Copa+ adds a rate adjustment factor θ to control the adjustment granularity of W, i.e., $\lambda = V \cdot \theta\delta^{-1}$, and adaptively change θ according to whether D_{trough} is close to 0, i.e., fulfills inequality (2). In this way, Copa+ can appropriately adjust the rate adjustment granularity θ without modifying the original control target δ^{-1}. The specific execution process of Copa+ is shown in **Algorithm 1**. Similar to Copa, on the receiving of each ACK, Copa+ updates delay signals and adjusts the sending rate according to the difference between the current rate and the target rate. Different from Copa, Copa+ set the adjustment granularity λ to be $V \cdot \theta \cdot \delta^{-1}$ rather than $V\delta^{-1}$, as shown in line 20. Wherein, the rate adjustment granularity θ is adjusted periodically according to a cubic function, as shown in lines 11-19. By the way, the entrance criterion of the competitive mode is also enhanced by utilizing the periodical behaviors in Copa+.

In detail, the cycle of adjusting θ is distinguished by the direction of queuing delay variation, similar to the oscillation cycle from *point B* to D in Fig. 1. Specifically, as shown in line 3 of the **Algorithm 1**, if the current d_q is larger than the history d_q, the variation direction is positive. Otherwise, the direction is negative. When the variation direction of d_q becomes negative from positive (or becomes positive from negative), the direction is considered as occurring a veer. Once d_q has continuously changed its direction twice, Copa+ judges that d_q has gone through an oscillation cycle. As shown in Fig. 1, d_q reaches D_{trough} when the queuing delay changes from decreasing to increasing.

The cubic function is utilized to guide the adjustment of θ due to the following reasons. On the one hand, θ can quickly approach to the inflection point, where the buffer occupancy can be cleared in the previous cycle, similar to the adjustment

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Copa+: on the receiving of each ACK

1: Update RTT_{min}, RTT_{max}, RTT_{stand} and d_q
2: Update V the same as Copa
3: $direction \leftarrow 1$ if $d_q > d_q^{pre}$ else -1
4: $cnt \leftarrow cnt + 1$ if $direction \cdot preDirection < 0$ else cnt
5: $d_q^{pre} \leftarrow d_q$; $preDirection \leftarrow direction$
6: **if** $d_q < 0.1(RTT_{max} - RTT_{min})$ **then**
7: $\quad nearEmpty \leftarrow True$
8: **else if** $cnt == 0$ **then**
9: $\quad$ Operations in the competitive mode the same as Copa
10: **end if**
11: **if** $cnt >= 2$ AND $V == 1$ **then**
12: $\quad$ **if** $nearEmpty$ **then**
13: $\quad\quad \theta_{empty} \leftarrow \theta$; $\theta \leftarrow \beta \cdot \theta$; $T_{empty} \leftarrow t$
14: $\quad$ **else**
15: $\quad\quad K \leftarrow [\theta_{empty} \cdot (1-\beta)/\omega]^{1/3}$
16: $\quad\quad \theta \leftarrow \omega[(t - T_{empty}) - K]^3 + \theta_{empty}$
17: $\quad$ **end if**
18: $\quad cnt \leftarrow 0$; $nearEmpty \leftarrow False$
19: **end if**
20: $W \leftarrow \begin{cases} W + V \cdot \theta \cdot \delta^{-1}/W, if\ W/RTT_{stand} \leq \delta^{-1} \cdot d_q \\ W - V \cdot \theta \cdot \delta^{-1}/W, if\ W/RTT_{stand} > \delta^{-1} \cdot d_q \end{cases}$

of the congestion window in TCP Cubic [3]. Even if the inflection point could not make D_{trough} close to 0 again, Copa+ would probe up a better value quickly for θ. On the other hand, Copa+ could ensure that the λ of different flows converge to the same value eventually with the same AIMD rules as TCP Cubic. Specifically, Copa+ seeks a proper λ via modulating θ as shown in lines 11-19 and Fig. 4. At the end of each cycle, if the minimum queuing delay D_{trough} during this cycle satisfies (2), Copa+ considers that D_{trough} has been close to 0. Thus, Copa+ decreases θ. Otherwise, Copa+ deems D_{trough} has not been close to 0 and hence increases θ. The detailed regulation system of θ is shown in Fig. 4. When D_{trough} has been close to 0 in the last cycle, Copa+ denotes the current time t as T_{empty}, and also denotes the current θ as θ_{empty} which will be the inflection point for the next modulation round of θ. Then, updating θ as $\beta \cdot \theta$ via multiplicative decrease, where β is set to 0.75 by experience. If D_{trough} has not been close to 0, θ is increased by a cubic function of the current time t:

$$\theta = \omega[(t - T_{empty}) - K]^3 + \theta_{empty} \tag{26}$$

Where w is a pre-configured parameter with default value 0.1 and K is

$$K = [\theta_{empty} \cdot (1-\beta)/\omega]^{1/3} \tag{27}$$

In this way, when t starts from T_{empty}, θ approaches to its target θ_{empty}. When $t - T_{empty}$ is close to K, θ grows gently around the inflection point, i.e., staying around θ_{empty} for a relatively long time. If D_{trough} has not been close to 0 in any cycle before θ is larger than θ_{empty}, the adjustment of θ will step into the detecting phase by following a convex profile. In this condition, the growth speed of θ keeps increasing until a new θ_{empty} makes D_{trough} close to 0. Note that the regulation of θ is effective without the assumption of synchronization, i.e., Copa+ can clear the buffer occupancy periodically with asynchronous flows.

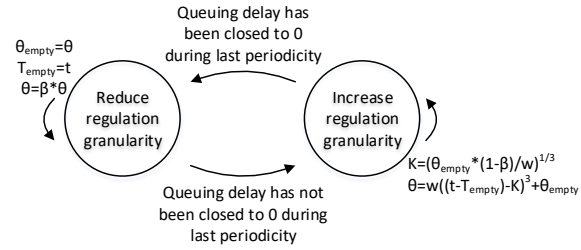


Fig. 4. State transition for the regulation of θ

In addition, we also optimize the entrance criterion of competitive mode. In detail, as shown in lines 7-11, we add constraints which deactivate the entrance of competitive mode under the condition of observing direction change. This optimization is simple but reasonable, because loss-based flows will keep injecting packets into the link until the buffer overflows. Correspondingly, d_q would keep increasing, and the direction change would be eliminated until packet loss occurs. In this way, compared with Copa, Copa+ will observe the direction change at packet loss and exit competitive mode, but soon enter back. This constraint makes Copa+ more moderate than Copa when it competes with loss-based flows.

All in all, Copa+ dynamically adjusts parameter θ, i.e., the adjustment granularity of W, to make D_{trough} satisfy inequality (2) periodically for the correct estimation of RTT_{base}. Moreover, Copa+ also enhances the entrance criterion of competitive mode. By the way, Copa+ inherits all the advantages of Copa because it doesn't change the core idea of Copa. In addition, compared to Copa, Copa+ only requires several extra variables and operations on the receiving of ACK and thus won't lead to large extra overhead.

V. Evaluation

To evaluate Copa+, we conduct simulations based on NS3 [24], as well as experiments based on a user-space implementation in Linux. The user-space implementation of Copa+ is extended from the Copa implementation provided by [9]. Specifically, the NS3 simulations verify our analysis, and the user-space implementation is executed over both emulated and real-world links to evaluate the performance of Copa+.

A. Estimation of RTT_{base}

To verify that Copa+ can address the issues of Copa by accurately measuring RTT_{base}, we repeat the NS3 simulations in Section III-C. For the convenience of observation, we also disable the competitive mode of Copa+. As shown in Fig. 3, different from Copa, which mistakenly estimates RTT_{base} with small C, small RTT_{base} or large N, Copa+ can always estimate RTT_{base} correctly no matter what the values of C, RTT_{base} and N are. This is mainly due to the adaptive regulation of parameter θ in Copa+. Correspondingly, Copa+

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE THROUGHPUT AND QUEUING DELAY RATIO OF COPA:COPA+

	C		N		RTT_{base}	
	$60Mbps$	$140Mbps$	5	40	$12ms$	$32ms$
Thr	1.001	1.001	1.000	1.001	1.001	1.005
Avg. d_q	1.27	0.97	1.00	1.29	1.31	1.04

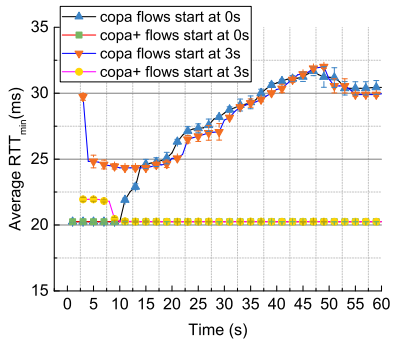


Fig. 5. Estimation of RTT_{base} with late-comers

achieves significant lower average queuing delay d_q when $C = 60Mbps$, $RTT_{base} = 12ms$ or $N = 40$, as shown in Table II. Meanwhile, Copa+ always keeps similar high throughput as Copa regardless of the values of C, RTT_{base} and N. It indicates that Copa+ doesn't suffer from any link under-utilization even if it clears the buffer occupancy periodically for the measurement of RTT_{base}. In other words, the regulation of parameter θ in Copa+ is suitable.

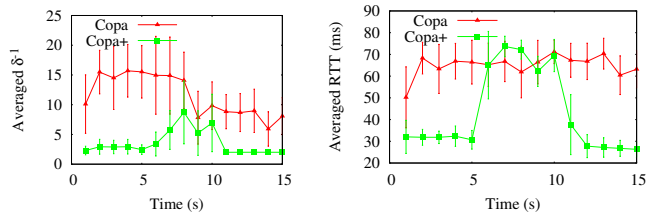
B. Fairness

To show how Copa+ performs with late-comers, we conduct simulations based on the above scenario, but let 40 initial flows start concurrently at $0s$ and 10 late-comers flows start at $3s$. We record the RTT_{min} upon receiving each ACK and calculate their average values in every second. As all the initial flows have similar RTT_{min} in every second, we show the average RTT_{min} of all 40 Copa initial flows every second in Fig. 5. Under this condition, the **Corollary 1** is fulfilled. Thus, Copa flows can not clear the buffer occupancy periodically. Accordingly, the average RTT_{min} increases to wrong values after $10s$. In the first $10s$, the average RTT_{min} equals to $RTT_{base} = 20ms$, because RTT_{min} couldn't be updated by a larger value until $10s$.

Similarly, the average RTT_{min} of the 10 Copa late-comers are also shown in Fig. 5. Because the buffer occupancy has not been cleared periodically, the Copa late-comers would observe a larger RTT_{min} at $3s$. The average RTT_{min} also increases to wrong values after $13s$. On the other side, with wrong RTT_{min}, the Copa late-comers would underestimate the queuing delay d_q and overestimate the target rate in the first $10s$. Consequently, the average throughput of the Copa late-comers is greatly larger than that of the Copa flows in the first $10s$, as shown in Table III. After $10s$, all Copa flows would

TABLE III
PERFORMANCE COMPARISON WITH LATE-COMERS FLOWS

	Avg. Thr. ratio of late-comers: initial flows	Avg. d_q
Copa	1.49 (first 10s); 1.06 (whole process)	18.78 ms
Copa+	0.87 (fisrt 10s); 0.98 (whole process)	15.01 ms



(a) Evolution of δ^{-1} (b) Evolution of RTT

Fig. 6. The competitive mode of Copa+ and Copa when Cubic flows enter and leave

eventually have the same wrong RTT_{min} and their throughput becomes similar to each other. In sum, Copa suffers from large queuing delay and unfairness under this condition.

In contrast, we repeat the simulations with Copa+. As shown in Fig. 5, Copa+ always maintains correct RTT_{min}. This is because Copa+ can adaptively adjust the parameter θ such that the buffer occupancy is emptied periodically for the correct measurement of RTT_{base}. Accordingly, all the late-comers can also correctly estimate RTT_{base}. Therefore, Copa+ can achieve lower queuing delay than Copa, as shown in Table III. Moreover, all the Copa+ flows are fair with the same RTT_{min}. The exception is that the Copa+ late-comers need time to adjust parameter θ for the correct measurement of RTT_{base} and suffer from lower average throughput than the initial Copa+ flows during the convergence process of θ.

C. Competitive Mode

To evaluate the impacts of the competitive mode, we conduct emulations over a $12Mbps$ link with $20ms$ RTT and 3 BDP buffer based on the user-space implementation [34] in Linux and Mahimahi [33]. We run 3 Copa/Copa+ flows concurrently from $0s$ to $15s$. Meanwhile, 3 Cubic flows enter this link at $5s$ and leave at $10s$. Because $\delta^{-1} > 2$ indicates that Copa/Copa+ is in competitive mode, we show the average values of δ^{-1} of all Copa/Copa+ flows in Fig. 6(a). Copa flows enter competitive mode no matter whether there are Cubic flows or not. The underlying reason is the fact that inequality (2) holds in this condition. Thus, Copa suffers from high queuing delay even if there are only Copa flows in the first and last $5s$, as shown in Fig. 6(b). In contrast, Copa+ barely enters the competitive mode in the first or last $5s$. This is because Copa+ can periodically clear the buffer occupancy for correct measurement of RTT_{base}. Meanwhile, Copa+ could correctly enter or exit the competitive mode when Cubic flows start or left. Consequently, Copa+ can achieve lower average queuing delay than Copa, as shown in Fig. 6(b).

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

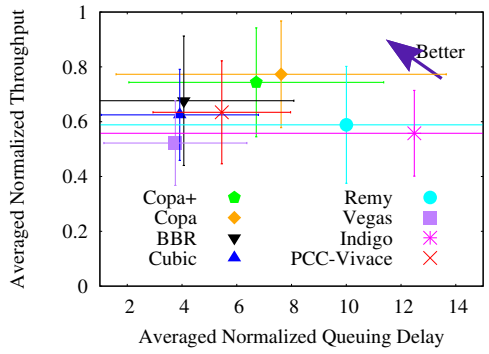


Fig. 7. Internet experimental results.The points denote the average queuing delay and the throughput normalized by the minimum queuing delay and the maximum throughput in each round of experiments, respectively. The lines denote the standard deviation.

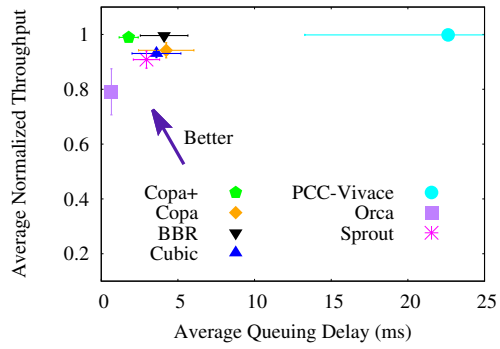


Fig. 8. Cellular experimental results. The points denote the average queuing delay and the throughput normalized by the maximum throughput observed at each link. The lines denote the standard deviation.

D. Real-world Evaluation

Subsequently, we compare Copa+ with existing CC schemes over the real-world Internet and emulated cellular links. The results confirm that Copa+ inherits the advantages of Copa and outperforms it in terms of low latency.

Internet. We deploy CC schemes in 4 cloud nodes located in Amsterdam, Frankfurt, Seoul and Toronto. The 6 links between each pair of these 4 nodes are either of short-distance, long-distance or transoceanic. In each node, we install the user-space implementation of Copa+ into Pantheon [23], an evaluation platform for academic research on CC. The compared CC schemes are taken from the third-party repositories of Pantheon and Linux kernel, including Copa [9], BBR [16], TCP Cubic [3], PCC Vivace [17], TCP Vegas [4], Remy [13], [14] and Indigo [23]. At each of the 6 links, two groups of experiments are conducted with each CC scheme: a) one flow lasts for $30s$; b)3 flows start at $0s$, $10s$ and $20s$, and all of them stop at $30s$. For each scenario, the experiments with different CC schemes are executed with the round robin pattern to remit the impacts of bandwidth variation in reality. For each round of experiments, we respectively normalize the throughput and the average queuing delay of each CC scheme with the maximum throughput and the minimum queuing delay observed in this round. Each round of experiments over each link is repeated 3 times. As different CC schemes have different performances over different links, we show the average normalized throughput and average normalized queuing delay in all the $2 \times 3 \times 6 = 36$ rounds of experiments in Fig. 7. The top left corner indicates a better performance, i.e., Copa, Copa+ and BBR perform well. In detail, Copa and Copa+ achieve higher link utilization than other schemes, similar to the results in [9], [18]. Moreover, compared to Copa, Copa+ gets a bit lower throughput due to the moderate competitive mode, as the loss-based CC schemes like Cubic coexist in the Internet links. Without loss-based CC schemes, Copa+ maintains similar throughput as Copa in Fig. 8 and Table II. What's more, Copa+ achieves 11.9% lower queuing delay than Copa because of the precise estimation of RTT_{base}.

Cellular. Finally, we evaluate Copa+ in Cellular network, which is challenging to CC due to the drastic bandwidth variation. We emulate cellular links with the traces provided by [18]. These data are collected from different scenarios in New York City, including taxi, home, bus and walking on Times Square. We respectively run 10 flows with different CC schemes over each link and the measured throughput is normalized by the maximum throughput observed at this link. The average normalized throughput and average queuing delay among all links are shown in Fig. 8. The top left corner indicates better performance. Copa+ achieves the highest average throughput and the second-lowest average queuing delay over all links. Specifically, Copa+ achieves 50.3%, 56.4%, 57.8% and 92.1% lower queuing delay than Cubic, BBR, Copa and PCC-Vivace, respectively. Copa+ also achieves 39.4% lower queuing delay and 8.9% higher link utilization compared to the Sprout [27], which is specially designed for cellular network.

VI. Conclusion

In this paper, we reveal that Copa can't clear the buffer occupancy periodically for correct measurement of RTT_{min} and thus suffers from large queuing delay and unfairness issues under certain conditions. Based on this theoretical result, we design Copa+, which enhances Copa by a parameter adaptation mechanism and an optimized competitive mode entrance criterion. Extensive simulations and experiments indicate that Copa+ can periodically clear the buffer occupancy and thus achieve a lower queuing delay and better fairness than Copa. Inheriting the advantages of Copa, Copa+ performs well under different network environments. Code of Copa+ is available at https://github.com/Wanchun-Jiang/genericCC

Acknowledgement

This work is partly sponsored by the National Natural Science Foundation of China (Grant No. 61972421, No.61902307), the Innovation-Driven Project of Central South University (No. 2020CX033), the Fundamental Research Funds for the Central Universities (No. xzy012020014) and the CCF-AFSG Research Fund. We appreciate the anonymous reviewers for their constructive comments.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] V. Jacobson, "Congestion Avoidance and Control," SIGCOMM, 1988

[2] K. Tan, J. Song, Q. Zhang, and M. Sridharan, "A Compound TCP Approach for High-speed and Long Distance Networks," INFOCOMM, 2006

[3] S. Ha, I. Rhee, and L. Xu, "A New TCP-Friendly High-Speed TCP Variant," ACM SIGOPS Operating System Review, 2008, vol. 42(5), pp. 64–74.

[4] L. S. Brakmo, S. W. O'Malley, and L. L. Peterson, "TCP Vegas: New Techniques for Congestion Detection and Avoidance," SIGCOMM, 1994.

[5] Handley, M, "Delay is not an option: Low latency routing in space," In Proceedings of the 17th ACM Workshop on Hot Topics in Networks, 2018, pp. 85–91.

[6] Alizadeh, M., Kabbani, A., Edsall, T., Prabhakar, B., Vahdat, A., and Yasuda, M, "Less is more: Trading a little bandwidth for ultra-low latency in the data center," NSDI, 2012.

[7] D. Wei, C. Jin, S. Low, and S. Hegde, "FAST TCP: Motivation, Architecture, Algorithms, Performance," IEEE/ACM Trans. on Networking, 2006, vol. 14(6), pp. 1246–1259.

[8] Gettys, Jim, "Bufferbloat: Dark Buffers in the Internet," Internet Computing, IEEE, 2011, vol. 15, pp. 96 - 96.

[9] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the internet," NSDI, 2018, pp. 329–342.

[10] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data Center TCP (DCTCP)," SIGCOMM, 2010, pp. 329–342.

[11] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," SIGCOMM, 2015.

[12] S. Shalunov, G. Hazel, J. Iyengar, and M. Kuehlewind, "Low Extra Delay Background Transport (LEDBAT)," RFC 6817 (Experimental), Internet Engineering Task Force, 2012.

[13] K. Winstein and H. Balakrishnan, "TCP ex Machina: Computer-Generated Congestion Control," SIGCOMM, 2013.

[14] A. Sivaraman, K. Winstein, P. Thaker, and H. Balakrishnan, "An Experimental Study of the Learnability of Congestion Control," SIGCOMM, 2014.

[15] R. Mittal, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, D. Zats, et al, "TIMELY: RTT-based Congestion Control for the Datacenter," SIGCOMM, 2015.

[16] Cardwell N , Cheng Y , Gunn C S , et al, "BBR: congestion-based congestion control," Communications of the ACM, 2017, vol. 60(2), pp. 58–66.

[17] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "Pcc vivace: Online-learning congestion control," NSDI, 2018.

[18] Abbasloo S , Yen C Y , Chao H J, "Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet," SIGCOMM, 2020.

[19] Tong Meng, Neta Rozen Schiff, P. Brighten Godfrey, and Michael Schapira, "PCC Proteus: Scavenger Transport And Beyond," SIGCOMM, 2020.

[20] Carlucci G , Cicco L D , Holmer S , et al, "Analysis and design of the google congestion control for web real-time communication (WebRTC)," The 7th International Conference. ACM, 2016.

[21] C. Lee, C. Park, K. Jang, S. Moon and D. Han, "DX: Latency-Based Congestion Control for Datacenters," IEEE/ACM Transactions on Networking, 2017, pp. 335–348.

[22] Gautam Kumar, Nandita Dukkipati, et al, "Swift: Delay is Simple and Effective for Congestion Control in the Datacenter," SIGCOMM, 2020, pp. 514–528.

[23] Levis, and Keith Winstein, "Pantheon: the training ground for Internet congestion-control research," , 2018, pp. 731–743.

[24] NSNAM, "Network Simulator 3," , 2021.

[25] Venkat Arun and Hari Balakrishnan, "Copa: Practical Delay-Based Congestion Control for the Internet," https://web.mit.edu/copa/, 2018.

[26] Nitin Garg, "Evaluating COPA congestion control for improved video performance," https://engineering.fb.com/2019/11/17/video-engineering/copa/, 2019.

[27] Keith Winstein, Anirudh Sivaraman, Hari Balakrishnan, et al. "Stochastic Forecasts Achieve High Throughput and Low Delay over Cellular Networks," NSDI, 2013, pp. 459–471.

[28] Zhang, Lixia and Shenker, Scott and Clark, Daivd D, "Observations on the dynamics of a congestion control algorithm: The effects of two-way traffic," Proceedings of the conference on Communications architecture & protocols, 1991, vol. 7(2), pp. 133–147.

[29] G. Carofiglio, L. Muscariello, D. Rossi, and S. Valenti, "The Quest for LEDBAT Fairness," IEEE Global Telecommunications Conference, 2010, pp. 1–6.

[30] S. Floyd and V. Jacobson, "Random early detection gateways for congestionavoidance," IEEE/ACM Transactions on Networking, 1993, vol. 1(4), pp. 397—413.

[31] S. Athuraliya, V. H. Li, S. H. Low, and Q. Yin, "Rem: Active queue management," IEEE Network, 2010.

[32] Hollot C V, Misra V, Towsley D, et al, "Analysis and design of controllers for AQM routers supporting TCP flows," IEEE Transactions on automatic control, 2002, vol. 47(6), pp. 945-959.

[33] R Netravali and A Sivaraman, et al, "Mahimahi: Accurate Record-and-Replay for HTTP," Proc. USENIX ATC, 2015, vol. 47(6), pp. 417–429.

[34] Venkat Arun, "GenericCC," Proc. USENIX ATC, 2021.

[35] Kefan Chen, Danfeng Shan, Xiaohui Luo, Tong Zhang, Yajun Yang, Fengyuan Ren, "One Rein to Rule Them All: A Framework for Datacenter-to-User Congestion Control", APNET, 2020.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:28:41 UTC from IEEE Xplore. Restrictions apply.